

Running jobs

This page is intended for the user who is already familiar with the concepts of job scheduling and job scripts, and who wants guidance on submitting jobs to our clusters. If you have not worked on a large shared computer cluster before, you should probably read **What is a scheduler?** first.

All jobs must be submitted via the scheduler!

Exceptions are made for compilation and other tasks not expected to consume more than about 10 CPU-minutes and about 4 gigabytes of RAM. Such tasks may be run on a login node. In no case should you run processes on compute nodes except via the scheduler.

On our clusters, the job scheduler is the **Slurm Workload Manager** (https://en.wikipedia.org/wiki/Slurm_Workload_Manager). Comprehensive **documentation for Slurm** (<https://slurm.schedmd.com/documentation.html>) is maintained by SchedMD. If you are coming to Slurm from PBS/Torque, SGE, LSF, or LoadLeveler, you might find this table of **corresponding commands** (<https://slurm.schedmd.com/rosetta.pdf>) useful.

Contents

Use sbatch to submit jobs

Memory

Use squeue or sq to list jobs**Where does the output go?****Accounts and projects****Examples of job scripts**

Serial job

Array job

Threaded or OpenMP job

MPI job

GPU job

Interactive jobs**Monitoring jobs****Cancelling jobs****Resubmitting jobs for long-running computations**

Restarting using job arrays

Resubmission from the job script

Automating job submission

Do not specify a partition

Cluster particularities

Troubleshooting

Avoid hidden characters in job scripts

Cancellation of jobs with dependency conditions which cannot be met

Job cannot load a module

Jobs inherit environment variables

Job hangs / no output / incomplete output

Job status and priority

Further reading

Use sbatch to submit jobs

The command to submit a job is **sbatch** (<https://slurm.schedmd.com/sbatch.html>):

```
$ sbatch simple_job.sh
Submitted batch job 123456
```

A minimal Slurm job script looks like this:

File : simple_job.sh

```
#!/bin/bash
#SBATCH --time=00:15:00
#SBATCH --account=def-someuser
echo 'Hello, world!'
sleep 30
```

On general-purpose (GP) clusters, this job reserves 1 core and 256MB of memory for 15 minutes. On **Trillium**, this job reserves the whole node with all its memory. Directives (or *options*) in the job script are prefixed with #SBATCH and must precede all executable commands. All available directives are described on the **sbatch page** (<https://slurm.schedmd.com/sbatch.html>). Our policies require that you supply at least a time limit (--time) for each job. You may also need to supply an account name (--account). See **Accounts and projects** below.

You can also specify directives as command-line arguments to sbatch. So for example,

```
$ sbatch --time=00:30:00 simple_job.sh
```

will submit the above job script with a time limit of 30 minutes. The acceptable time formats include "minutes", "minutes:seconds", "hours:minutes:seconds", "days-hours", "days-hours:minutes" and "days-hours:minutes:seconds". Please note that the time limit will strongly affect how quickly the job is started, since longer jobs are **eligible to run on fewer nodes**.

Please be cautious if you use a script to submit multiple Slurm jobs in a short time. Submitting thousands of jobs at a time can cause Slurm to become **unresponsive** to other users. Consider using an **array job** instead, or use `sleep` to space out calls to `sbatch` by one second or more.

Memory

Memory may be requested with `--mem-per-cpu` (memory per core) or `--mem` (memory per node). On general-purpose (GP) clusters, a default memory amount of 256 MB per core will be allocated unless you make some other request. On **Trillium**, only whole nodes are allocated along with all available memory, so a memory specification is not required there.

A common source of confusion comes from the fact that some memory on a node is not available to the job (reserved for the OS, etc.). The effect of this is that each node type has a maximum amount available to jobs; for instance, nominally "128G" nodes are typically configured to permit 125G of memory to user jobs. If you request more memory than a node-type provides, your job will be constrained to run on higher-memory nodes, which may be fewer in number.

Adding to this confusion, Slurm interprets K, M, G, etc., as **binary prefixes** (https://en.wikipedia.org/wiki/Binary_prefix), so `--mem=125G` is equivalent to `--mem=128000M`. See the *Available memory* column in the *Node characteristics* table for each GP cluster for the Slurm specification of the maximum memory you can request on each node: **Fir**, **Narval**, **Nibi**, **Rorqual**.

Use `squeue` or `sq` to list jobs

The general command for checking the status of Slurm jobs is `squeue`, but by default it supplies information about **all** jobs in the system, not just your own. You can use the shorter `sq` to list only your own jobs:

```
$ sq
JOBID   USER   ACCOUNT   NAME   ST   TIME_LEFT  NODES  CPUS   GRES  MIN_MEM  NODELIST (REASON)
123456  smithj  def-smithj simple_j R    0:03       1     1   (null)    4G cdr234 (None)
123457  smithj  def-smithj bigger_j PD   2-00:00:00  1    16   (null)   16G (Priority)
```

The ST column of the output shows the status of each job. The two most common states are PD for *pending* or R for *running*.

If you want to know more about the output of `sq` or `squeue`, or learn how to change the output, see the **online manual page for `squeue`** (<https://slurm.schedmd.com/squeue.html>). `sq` is a local customization.

Do not run `sq` or `squeue` from a script or program at high frequency (e.g. every few seconds). Responding to `squeue` adds load to Slurm, and may interfere with its performance or correct operation. See **Email notification** below for a much better way to learn when your job starts or ends.

Where does the output go?

By default the output is placed in a file named "slurm-", suffixed with the job ID number and ".out" (e.g. `slurm-123456.out`), in the directory from which the job was submitted. Having the job ID as part of the file name is convenient for troubleshooting.

A different name or location can be specified if your workflow requires it by using the `--output` directive. Certain replacement symbols can be used in a filename specified this way, such as the job ID number, the job name, or the **job array** task ID. See the **vendor documentation on sbatch** (<https://slurm.schedmd.com/sbatch.html>) for a complete list of replacement symbols and some examples of their use.

Error output will normally appear in the same file as standard output, just as it would if you were typing commands interactively. If you want to send the standard error channel (stderr) to a separate file, use `--error`.

Accounts and projects

Every job must have an associated account name corresponding to a **Resource Allocation Project** (RAP). If you are a member of only one account, the scheduler will automatically associate your jobs with that account.

If you receive one of the following messages when you submit a job, then you have access to more than one account:

```
You are associated with multiple _cpu allocations...
Please specify one of the following accounts to submit this job:
```

```
You are associated with multiple _gpu allocations...
Please specify one of the following accounts to submit this job:
```

In this case, use the `--account` directive to specify one of the accounts listed in the error message, e.g.:

```
#SBATCH --account=def-user-ab
```

To find out which account name corresponds to a given Resource Allocation Project, log in to **CCDB** (<https://ccdb.alliancecan.ca>) and click on *My Projects -> My Resources and Allocations*. You will see a list of all the projects you are a member of. The string you should use with the `--account` for a given project is under the column *Group Name*. Note that a Resource Allocation Project may only apply to a specific cluster (or set of clusters) and therefore may not be transferable from one cluster to another.

In the illustration below, jobs submitted with `--account=def-fuenma` will be accounted against RAP zh-914-aa

My Resources and Allocations

Computational resources are made available to research groups through Resource Allocation Projects (RAP). This page shows the resources and allocations that you have access to as an owner, manager or member of any RAP. Each RAP is identified by a RAPI (e.g., *abc-123-ab*) and an associated group name (e.g *def-[profname][-xx]*).

RAP Owners and Managers can view and click on the **Group Name** link to manage RAP membership.

When available, click on the link in the **Allocations ...** column to view allocation details.

Opportunistic Use indicates that you can use compute (CPU and GPU) with this project. Jobs submitted with this project is scheduled with low **priority**. While this should be enough to fulfill modest compute needs, there is no guarantee of how much resources can be consumed with these projects.

While you can also compute on Niagara with your Default RAP, you need to request access to this cluster. Go to this [page](#), and click on "Join" next to Niagara and Mist. Note that there is no default storage allocation on Niagara.

Project Identifier Groupname[RAPI] (Owner)	Allocations (as owner or member of the RAP)	Opportunistic Use
<i>Beluga</i>		
def-fuenma [zhf-914-aa]	1 TB Project Storage	Compute (CPU, GPU)
def-fuenma-ab [zhf-914-ab]	1 TB Project Storage	Compute (CPU, GPU)
<i>Cedar</i>		
def-fuenma [zhf-914-aa]	1 TB Project Storage	Compute (CPU, GPU)
def-fuenma-ab [zhf-914-ab]	1 TB Project Storage	Compute (CPU, GPU)
<i>Graham</i>		
def-fuenma [zhf-914-aa]	1 TB Project Storage	Compute (CPU, GPU)

Finding the group name for a Resource Allocation Project (RAP)

If you plan to use one account consistently for all jobs, once you have determined the right account name you may find it convenient to set the following three environment variables in your `~/ .bashrc` file:

```
export SLURM_ACCOUNT=def-someuser
export SBATCH_ACCOUNT=$SLURM_ACCOUNT
export SALLOC_ACCOUNT=$SLURM_ACCOUNT
```

Slurm will use the value of `SBATCH_ACCOUNT` in place of the `--account` directive in the job script. Note that even if you supply an account name inside the job script, *the environment variable takes priority*. In order to override the environment variable, you must supply an account name as a command-line argument to `sbatch`.

`SLURM_ACCOUNT` plays the same role as `SBATCH_ACCOUNT`, but for the `srun` command instead of `sbatch`. The same idea holds for `SALLOC_ACCOUNT`.

Examples of job scripts

Serial job

A serial job is a job which only requests a single core. It is the simplest type of job. The "simple_job.sh" which appears above in [Use sbatch to submit jobs](#) is an example.

Array job

Also known as a *task array*, an array job is a way to submit a whole set of jobs with one command. The individual jobs in the array are distinguished by an environment variable, `$SLURM_ARRAY_TASK_ID`, which is set to a different value for each instance of the job. The following example will create 10 tasks,

with values of `$SLURM_ARRAY_TASK_ID` ranging from 1 to 10:

File : array_job.sh

```
#!/bin/bash
#SBATCH --account=def-someuser
#SBATCH --time=0-0:5
#SBATCH --array=1-10
./myapplication $SLURM_ARRAY_TASK_ID
```

For more examples, see **Job arrays**. See **Job Array Support (https://slurm.schedmd.com/job_array.html)** for detailed documentation.

Threaded or OpenMP job

This example script launches a single process with eight CPU cores. Bear in mind that for an application to use OpenMP it must be compiled with the appropriate flag, e.g. `gcc -fopenmp ...` or `icc -openmp ...`

File : openmp_job.sh

```
#!/bin/bash
#SBATCH --account=def-someuser
#SBATCH --time=0-0:5
#SBATCH --cpus-per-task=8
export OMP_NUM_THREADS=$SLURM_CPUS_PER_TASK
./ompHello
```

For more on writing and running parallel programs with OpenMP, see **OpenMP**.

MPI job

This example script launches four MPI processes, each with 1024 MB of memory. The run time is limited to 5 minutes.

File : mpi_job.sh

```
#!/bin/bash
#SBATCH --account=def-someuser
#SBATCH --ntasks-per-node=4      # number of MPI processes
#SBATCH --nodes=1
```

```
#SBATCH --mem-per-cpu=1024M    # memory; default unit is megabytes
#SBATCH --time=0-00:05        # time (DD-HH:MM)
srun ./mpi_program            # mpirun or mpiexec also work
```

Large MPI jobs may span more than one node. Hybrid MPI/threaded jobs are also possible. For more on these and other options relating to distributed parallel jobs, see [Advanced MPI scheduling](#).

GPU job

Please see [Using GPUs with Slurm](#) for a discussion and examples of how to request GPU resources.

Interactive jobs

Though batch submission is the most common and most efficient way to take advantage of our clusters, interactive jobs are also supported. These can be useful for things like:

- Data exploration at the command line
- Interactive console tools like R and iPython
- Significant software development, debugging, or compiling

You can start an interactive session on a compute node with **salloc** (<https://slurm.schedmd.com/salloc.html>). In the following example we request one task, which corresponds to one CPU cores and 3 GB of memory, for an hour:

```
$ salloc --time=1:0:0 --mem-per-cpu=3G --ntasks=1 --account=def-someuser
salloc: Granted job allocation 1234567
$ ...          # do some work
$ exit         # terminate the allocation
salloc: Relinquishing job allocation 1234567
```

It is also possible to run graphical programs interactively on a compute node by adding the **--x11** flag to your **salloc** command. In order for this to work, you must first connect to the cluster with X11 forwarding enabled (see the [SSH](#) page for instructions on how to do that). Note that an interactive job with a duration of three hours or less will likely start very soon after submission as we have dedicated test nodes for jobs of this duration. Interactive jobs that request more than three hours run on the cluster's regular set of nodes and may wait for many hours or even days before starting, at an unpredictable (and possibly inconvenient) hour.

Monitoring jobs

See [Monitoring jobs](#).

Cancelling jobs

Use **scancel** (<https://slurm.schedmd.com/scancel.html>) with the job ID to cancel a job:

```
$ scancel <jobid>
```

You can also use it to cancel all your jobs, or all your pending jobs:

```
$ scancel -u $USER
$ scancel -t PENDING -u $USER
```

Resubmitting jobs for long-running computations

When a computation is going to require a long time to complete, so long that it cannot be done within the time limits on the system, the application you are running must support **checkpointing**. The application should be able to save its state to a file, called a *checkpoint file*, and then it should be able to restart and continue the computation from that saved state.

For many users restarting a calculation will be rare and may be done manually, but some workflows require frequent restarts. In this case some kind of automation technique may be employed.

Here are two recommended methods of automatic restarting:

- Using SLURM **job arrays**.
- Resubmitting from the end of the job script.

Our [Machine Learning tutorial](#) covers [resubmitting for long machine learning jobs](#).

Restarting using job arrays

Using the `--array=1-100%10` syntax one can submit a collection of identical jobs with the condition that only one job of them will run at any given time. The script should be written to ensure that the last checkpoint is always used for the next job. The number of restarts is fixed by the `--array` argument.

Consider, for example, a molecular dynamics simulations that has to be run for 1 000 000 steps, and such simulation does not fit into the time limit on the cluster. We can split the simulation into 10 smaller jobs of 100 000 steps, one after another.

An example of using a job array to restart a simulation:

File : job_array_restart.sh

```
#!/bin/bash
# -----
# SLURM script for a multi-step job on our clusters.
# -----
#SBATCH --account=def-someuser
#SBATCH --cpus-per-task=1
#SBATCH --time=0-10:00
#SBATCH --mem=100M
#SBATCH --array=1-10%1 # Run a 10-job array, one job at a time.
# -----
echo "Current working directory: `pwd`"
echo "Starting run at: `date`"
# -----
echo ""
echo "Job Array ID / Job ID: $SLURM_ARRAY_JOB_ID / $SLURM_JOB_ID"
echo "This is job $SLURM_ARRAY_TASK_ID out of $SLURM_ARRAY_TASK_COUNT jobs."
echo ""
# -----
# Run your simulation step here...

if test -e state.cpt; then
    # There is a checkpoint file, restart;
    mdrun --restart state.cpt
```



```

else
    # There is no checkpoint file, start a new simulation.
    mdrun
fi

# -----
echo "Job finished with exit code $? at: `date`"
# -----

```

Resubmission from the job script

In this case one submits a job that runs the first chunk of the calculation and saves a checkpoint. Once the chunk is done but before the allocated run-time of the job has elapsed, the script checks if the end of the calculation has been reached. If the calculation is not yet finished, the script submits a copy of itself to continue working.

An example of a job script with resubmission:

File : job_resubmission.sh

```

#!/bin/bash
# -----
# SLURM script for job resubmission on our clusters.
# -----
#SBATCH --job-name=job_chain
#SBATCH --account=def-someuser
#SBATCH --cpus-per-task=1
#SBATCH --time=0-10:00
#SBATCH --mem=100M
# -----
echo "Current working directory: `pwd`"
echo "Starting run at: `date`"
# -----
# Run your simulation step here...

if test -e state.cpt; then
    # There is a checkpoint file, restart;
    mdrun --restart state.cpt
else
    # There is no checkpoint file, start a new simulation.
    mdrun
fi

# Resubmit if not all work has been done yet.
# You must define the function work_should_continue().
if work_should_continue; then
    sbatch ${BASH_SOURCE[0]}
fi

# -----
echo "Job finished with exit code $? at: `date`"
# -----

```

Please note: The test to determine whether to submit a follow-up job, abbreviated as `work_should_continue` in the above example, should be a *positive test*. There may be a temptation to test for a stopping condition (e.g. is some convergence criterion met?) and submit a new job if the

condition is *not* detected. But if some error arises that you didn't foresee, the stopping condition might never be met and your chain of jobs may continue indefinitely, doing nothing useful.

Automating job submission

As described earlier, **array jobs** can be used to automate job submission. We provide a few other (more advanced) tools designed to facilitate running a large number of related serial, parallel, or GPU calculations. This practice is sometimes called *farming*, *serial farming*, or *task farming*. In addition to automating the workflow, these tools can also improve computational efficiency by bundling up many short computations into fewer tasks of longer duration.

The following tools are available on our clusters:

- [META-Farm](#)
- [GNU Parallel](#)
- [GLOST](#)

Do not specify a partition

Certain software packages such as **Masurca** (<https://github.com/alekseyzimin/masurca>) operate by submitting jobs to Slurm automatically, and expect a partition to be specified for each job. This is in conflict with what we recommend, which is that you should allow the scheduler to assign a partition to your job based on the resources it requests. If you are using such a piece of software, you may configure the software to use `--partition=default`, which the script treats the same as not specifying a partition.

Cluster particularities

There are certain differences in the job scheduling policies from one of our clusters to another and these are summarized by tab in the following section:

Beluga, Fir, Narval, Nibi, Rorqual **Trillium**

On these clusters, no jobs are permitted longer than 168 hours (7 days) and there is a limit of 1000 jobs, queued and running, per user. Production jobs should have a duration of at least an hour.

Troubleshooting

Avoid hidden characters in job scripts

Preparing a job script with a word processor instead of a text editor is a common cause of trouble. Best practice is to prepare your job script on the cluster using an editor such as nano, vim, or emacs. If you prefer to prepare or alter the script off-line, then:

- **Windows users:**
 - Use a text editor such as Notepad or Notepad++ (<https://notepad-plus-plus.org/>).
 - After uploading the script, use `dos2unix` to change Windows end-of-line characters to Linux end-of-line characters.
- **Mac users:**

- Open a terminal window and use an editor such as nano, vim, or emacs.

Cancellation of jobs with dependency conditions which cannot be met

A job submitted with `--dependency=afterok:<jobid>` is a *dependent job*. A dependent job will wait for the parent job to be completed. If the parent job fails (that is, ends with a non-zero exit code) the dependent job can never be scheduled and so will be automatically cancelled. See **sbatch** (https://slurm.schedmd.com/sbatch.html#OPT_dependency) for more on dependency.

Job cannot load a module

It is possible to see an error such as:

```
Lmod has detected the following error: These module(s) exist but cannot be
loaded as requested: "<module-name>/<version>"
Try: "module spider <module-name>/<version>" to see how to load the module(s).
```

This can occur if the particular module has an unsatisfied prerequisite. For example

```
$ module load gcc
$ module load quantumpresso/6.1
Lmod has detected the following error: These module(s) exist but cannot be loaded as requested: "quantumpresso/6.1"
Try: "module spider quantumpresso/6.1" to see how to load the module(s).
$ module spider quantumpresso/6.1

-----
quantumpresso: quantumpresso/6.1
-----
Description:
  Quantum ESPRESSO is an integrated suite of computer codes for electronic-structure calculations and materials
  modeling at the nanoscale. It is based on density-functional theory, plane waves, and pseudopotentials (both
  norm-conserving and ultrasoft).

Properties:
  Chemistry libraries/apps / Logiciels de chimie

You will need to load all module(s) on any one of the lines below before the "quantumpresso/6.1" module is available
to load.

  nixpkgs/16.09  intel/2016.4  openmpi/2.1.1

Help:

Description
=====
Quantum ESPRESSO is an integrated suite of computer codes
for electronic-structure calculations and materials modeling at the nanoscale.
It is based on density-functional theory, plane waves, and pseudopotentials
(both norm-conserving and ultrasoft).

More information
=====
- Homepage: http://www.pwscf.org/
```

In this case adding the line `module load nixpkgs/16.09 intel/2016.4 openmpi/2.1.1` to your job script before loading `quantumpresso/6.1` will solve the problem.

Jobs inherit environment variables

By default a job will inherit the environment variables of the shell where the job was submitted. The **module** command, which is used to make various software packages available, changes and sets environment variables. Changes will propagate to any job submitted from the shell and thus could affect the job's ability to load modules if there are missing prerequisites. It is best to include the line `module purge` in your job script before loading all the required modules to ensure a consistent state for each job submission and avoid changes made in your shell affecting your jobs.

Inheriting environment settings from the submitting shell can sometimes lead to hard-to-diagnose problems. If you wish to suppress this inheritance, use the `--export=none` directive when submitting jobs.

Job hangs / no output / incomplete output

Sometimes a submitted job writes no output to the log file for an extended period of time, looking like it is hanging. A common reason for this is the aggressive **buffering** performed by the Slurm scheduler, which will aggregate many output lines before flushing them to the log file. Often the output file will only be written after the job completes; and if the job is cancelled (or runs out of time), part of the output may be lost. If you wish to monitor the progress of your submitted job as it runs, consider running an **interactive job**. This is also a good way to find how much time your job needs.

Job status and priority

- For a discussion of how job priority is determined and how things like time limits may affect the scheduling of your jobs, see Job scheduling policies.
- If jobs *within your research group* are competing with one another, please see Managing Slurm accounts.

Further reading

- Comprehensive documentation (<https://slurm.schedmd.com/documentation.html>) is maintained by SchedMD, as well as some tutorials (<https://slurm.schedmd.com/tutorials.html>).
 - sbatch (<https://slurm.schedmd.com/sbatch.html>) command options
- There is also a "Rosetta stone" (<https://slurm.schedmd.com/rosetta.pdf>) mapping commands and directives from PBS/Torque, SGE, LSF, and LoadLeveler, to SLURM.
- Here is a text tutorial from CÉCI (http://www.ceci-hpc.be/slurm_tutorial.html), Belgium
- Here is a rather minimal text tutorial from Bright Computing (<http://www.brightcomputing.com/blog/bid/174099/slurm-101-basic-slurm-usage-for-linux-clusters>)

Retrieved from "https://docs.alliancecan.ca/mediawiki/index.php?title=Running_jobs&oldid=196837"

This page was last edited on 16 December 2025, at 20:07.